

PROJECT AND TEAM INFORMATION

Project Title

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Student / Team Information

<i>Team Name:</i> <i>Team #:</i>	<i>Blaze</i> <i>DSCPP-III-2026-T296</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture)</i>	<i>Gangwar, Abhinesh – 2510370192</i> <i>2510370192@geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture)</i>	<i>Jain, Sparsh – 2510370159</i> <i>2510370159@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture)</i>	<i>Bajaj, Krish – 2510370469</i> <i>2510370469@geu.ac.in</i> 
Team member 4 <i>(Last Name, name: student ID: email, picture)</i>	<i>Joshi, Snehil – 2510370146</i> <i>2510370146@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

During programming work, files are modified many times. Students often save multiple copies of the same project folder with names such as “final”, “final_new”, or “final_latest”. This method becomes confusing over time because it is difficult to identify the correct version, compare changes, or restore work after an error.

Version-control software solves this problem by recording important stages of a project. However, professional tools can feel difficult for beginners because they contain many commands and advanced concepts. MiniGit is designed as a smaller local system that explains the core idea in a simpler form. The application will let a user create a repository, add files for tracking, save file versions with commit messages, view previous commits, and restore an earlier version when necessary. It is meant as a learning tool, not as a replacement for full-scale version-control platforms.

By building MiniGit, the team will apply C++ programming, OOP, file handling, and data structures in one meaningful application. The project demonstrates how basic programming ideas can be combined to create a useful software tool.

State of the Art / Current Solution

Tools such as Git, Mercurial, and SVN are used to manage file versions in software projects. They preserve project history and provide features such as commits, branches, merging, collaboration, remote repositories, and recovery from mistakes. Git is especially common in modern development because it records changes in an organized and reliable manner.

For a beginner or for a small individual project, these tools may initially appear complicated. Their large command set and technical terminology can make it difficult to understand the basic workflow. As a result, many students still maintain versions manually by duplicating folders and renaming them repeatedly.

MiniGit focuses on a much smaller workflow: create a local repository, track selected files, save versions, inspect history, and restore a previous version. This reduced scope makes the main principles of version control easier to observe and understand without the complexity of a full professional tool.

Project Goals and Milestones

MiniGit aims to build a compact command-line version-control system for local text files. It will use C++, object-oriented design, data structures, and file handling to support essential version-management tasks.

Phase 1: Planning and Design: *Decide the feature scope, terminal commands, repository-folder structure, class responsibilities, selected data structures, and overall architecture. The basic system will support one user, one local repository, and a linear commit history. The command flow for `init`, `add`, `commit`, `log`, `checkout`, and `rollback` will be finalized.*

Phase 2: Repository and Commit Module: *Build `init` for repository setup, `add` for tracking files, `commit` for storing versions, and `log` for viewing history. The application will create a `.minigit` directory and save metadata and snapshots locally. Each commit will store an ID, message, timestamp, parent link, and file-snapshot details. A linked list will hold the commit sequence, and a hash table will speed up file and commit lookup.*

Phase 3: Restore, Test, and Document: *Develop `checkout` to recover a selected version and `rollback` to undo the latest restoration, using stack to store recent states. In this phase we will focus on validation, error handling, testing, documentation, a sample repository, and a demonstration. Optional features include simple branching and file-version comparison which depend on feasibility and time constraints during that stage.*

Project Approach

MiniGit will be a modular C++ terminal application with separate command, repository, and storage components.

Classes:-

- *Repository* handles initialization, file tracking, commits, history, and storage;
- *Commit* stores commit ID, message, timestamp, parent reference, and snapshots;
- *FileSnapshot* stores file-version details and snapshot location;
- *CommandHandler* processes terminal commands and validates arguments.

Data Structures:-

- *a singly linked list maintains commit history;*
- *a hash table enables quick file and commit lookup;*
- *a stack supports rollback after checkout; and file handling stores metadata and snapshots in `.minigit`.*

The core project will have one local branch and linear history. Tree or graph-based branch relationships will be attempted only as an optional extension after the required functionality is stable.

System Architecture (High Level Diagram)

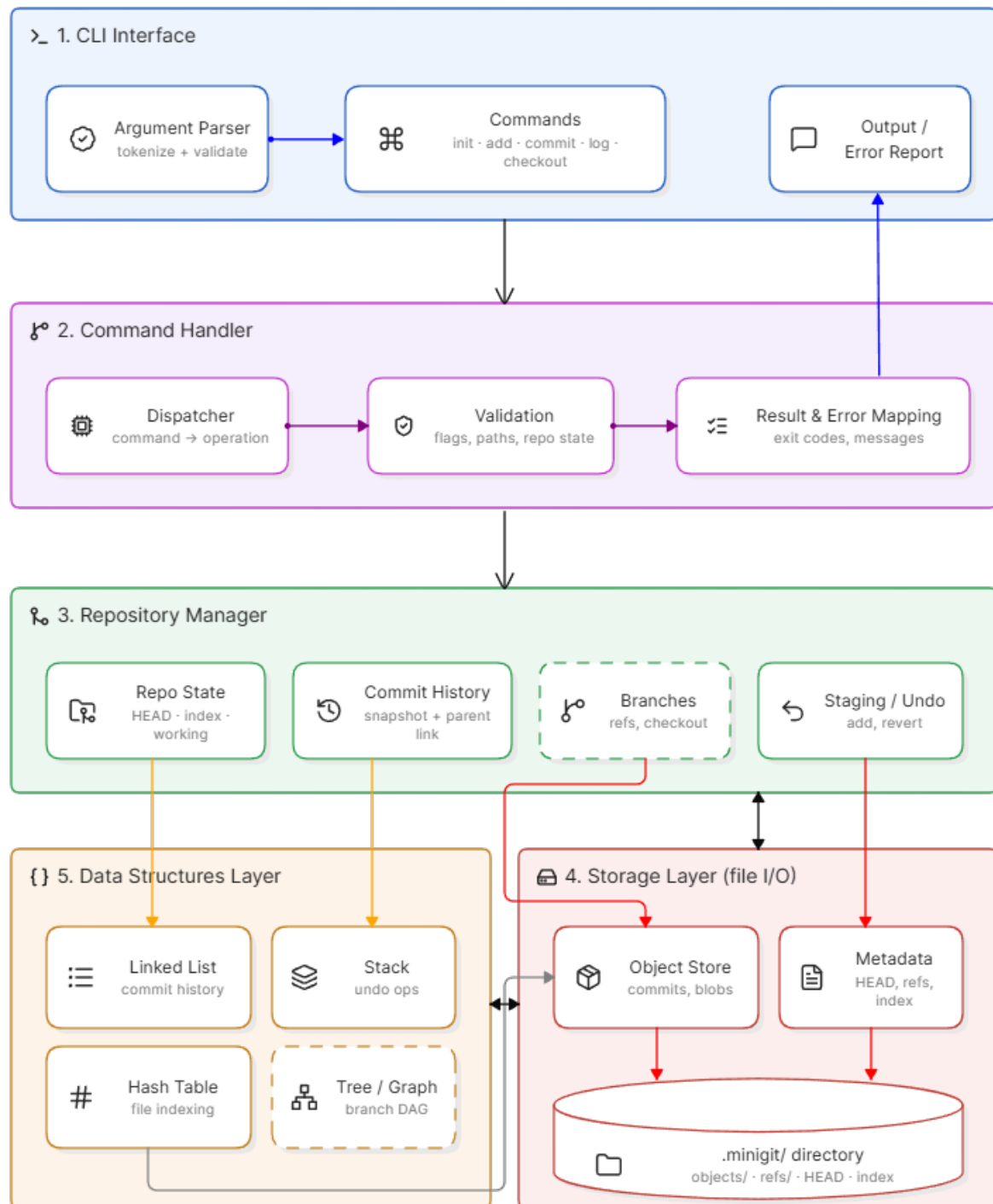


Figure 1: High-Level Architecture of MiniGit

Project Outcome / Deliverables

The final outcome will be a functional terminal-based MiniGit application written in C++. The core application will provide the following operations:

- ***init** — create a local repository.*
- ***add** — select files for tracking.*
- ***commit** — save the current tracked-file version.*
- ***log** — display the sequence of saved commits.*
- ***checkout** — restore a selected saved version.*
- ***rollback** — reverse the most recent restoration.*

The submission will include organized C++ source code, a sample repository, test cases, a README or user guide with command examples, a project report, and a live demonstration. Optional work may include basic local branching and a file-version comparison feature.

Assumptions

MiniGit is intended for a single user working on one local computer. It will handle text-based files such as C++ source files, header files, and text documents located inside the repository folder. The user is assumed to have permission to read and write these files. The project requires a standard C++ environment with file-system and file-I/O support. Remote repositories, collaboration, authentication, merging, conflict resolution, binary-file optimization, and other advanced Git features are outside the planned scope.

References

[1] Git SCM. Git Documentation. <https://git-scm.com/doc>

[2] Scott Chacon and Ben Straub. Pro Git, 2nd Edition. <https://git-scm.com/book/en/v2>

[3] Git SCM. git-checkout Documentation. <https://git-scm.com/docs/git-checkout>

[4] cppreference.com. C++ Standard Library Reference. <https://en.cppreference.com/>

[5] Pat Morin. Open Data Structures (in C++). <https://opendatastructures.org/ods-cpp/>

[6] NPTEL, IIT Delhi. Introduction to Data Structures and Algorithms. <https://nptel.ac.in/courses/106102064>